The normal distribution

In this lab, you'll investigate the probability distribution that is most central to statistics: the normal distribution. If you are confident that your data are nearly normal, that opens the door to many powerful statistical methods. Here we'll use the graphical tools of R to assess the normality of our data and also learn how to generate random numbers from a normal distribution.

Getting Started

Load packages

In this lab, we will explore and visualize the data using the **tidyverse** suite of packages as well as the **openintro** package.

Let's load the packages.

```
library(tidyverse)
library(openintro)
```

The data

This week you'll be working with fast food data. This data set contains data on 515 menu items from some of the most popular fast food restaurants worldwide. Let's take a quick peek at the first few rows of the data.

Either you can use `glimpse` like before, or `head` to do this.

```
library(tidyverse)
library(openintro)
data("fastfood", package='openintro')
head(fastfood)
```

```
## # A tibble: 6 x 17
##   restaurant item       calories cal_fat total_fat sat_fat trans_fat cholesterol
##   <chr>      <chr>         <dbl>   <dbl>     <dbl>   <dbl>     <dbl>       <dbl>
## 1 Mcdonalds  Artisan G~      380      60         7       2       0            95
## 2 Mcdonalds  Single Ba~      840     410        45      17       1.5         130
## 3 Mcdonalds  Double Ba~     1130     600        67      27       3           220
## 4 Mcdonalds  Grilled B~      750     280        31      10       0.5         155
## 5 Mcdonalds  Crispy Ba~      920     410        45      12       0.5         120
## 6 Mcdonalds  Big Mac         540     250        28      10       1            80
## # i 9 more variables: sodium <dbl>, total_carb <dbl>, fiber <dbl>, sugar <dbl>,
## #   protein <dbl>, vit_a <dbl>, vit_c <dbl>, calcium <dbl>, salad <chr>
```

You'll see that for every observation there are 17 measurements, many of which are nutritional facts.

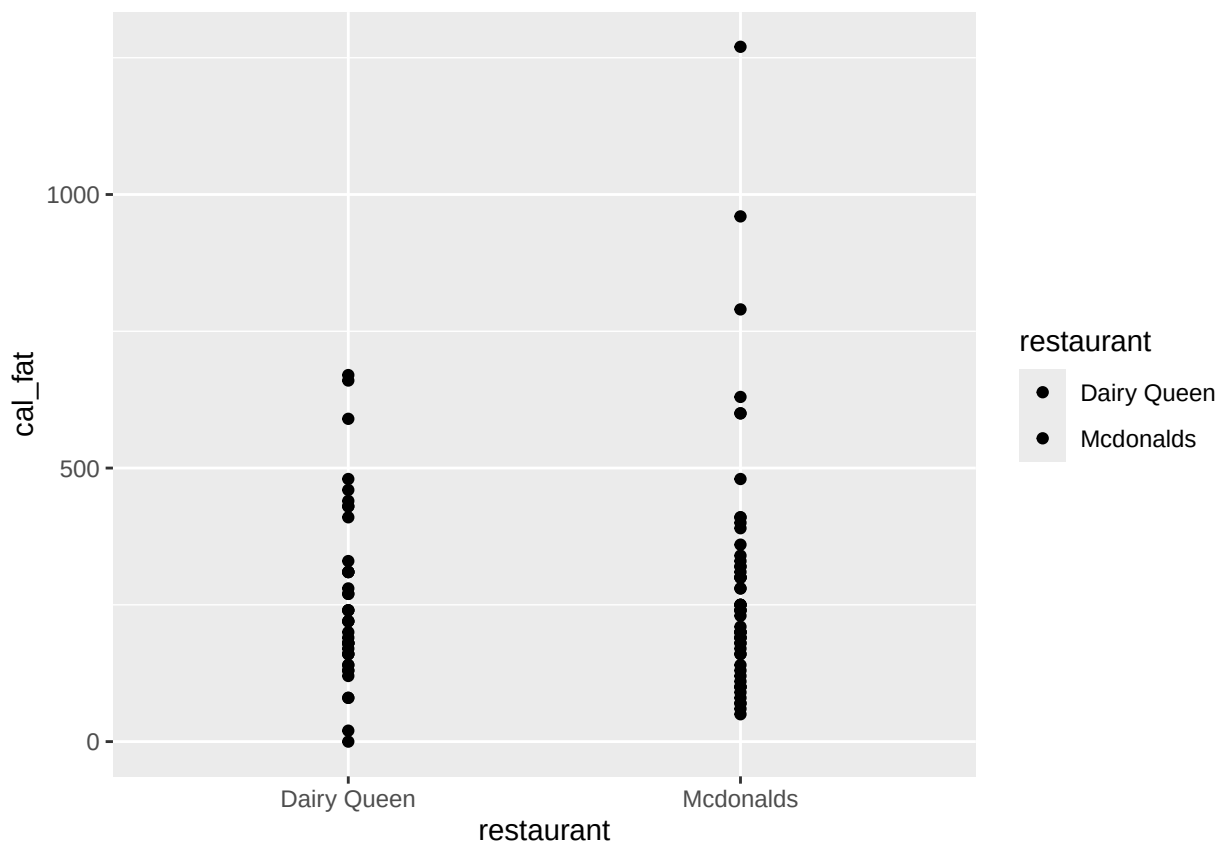
You'll be focusing on just three columns to get started: restaurant, calories, calories from fat.

Let's first focus on just products from McDonalds and Dairy Queen.

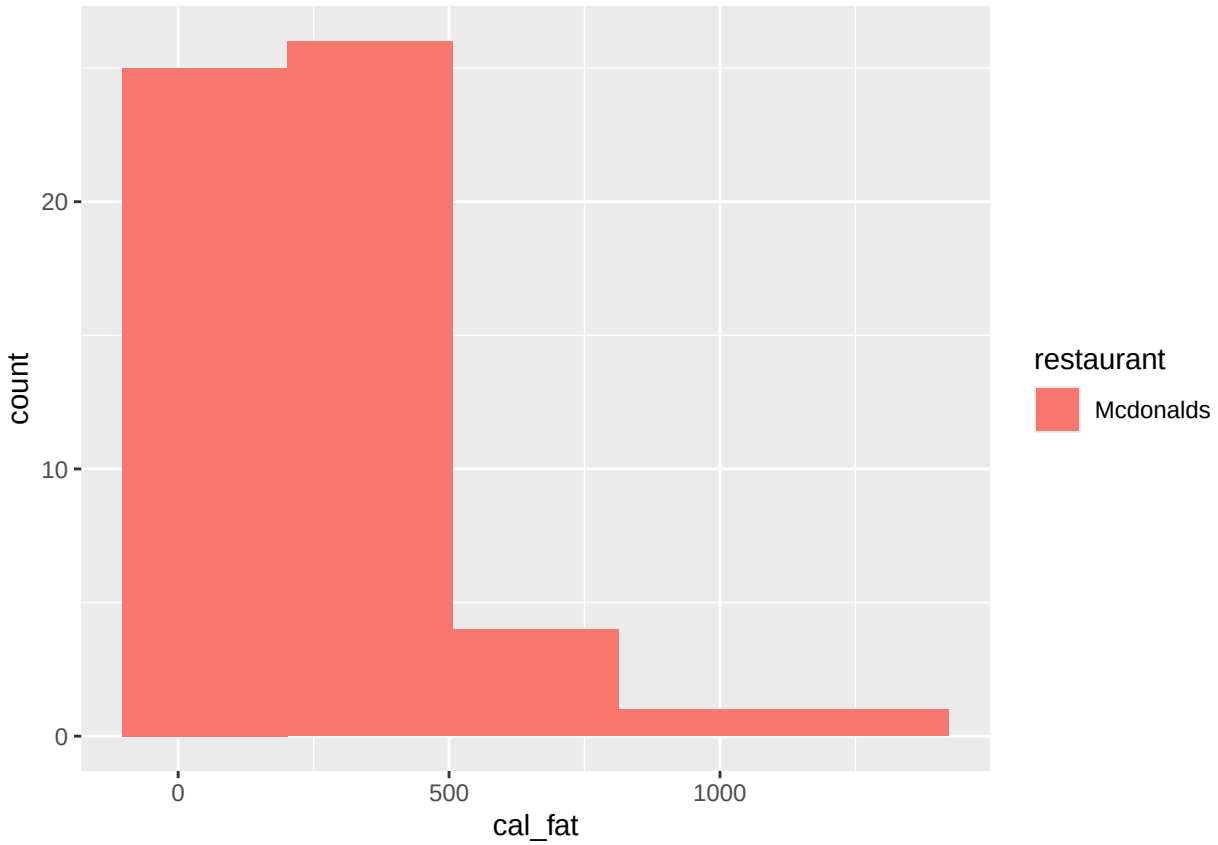
```r
mcdonalds <- fastfood %>%
  filter(restaurant == "Mcdonalds")
dairy_queen <- fastfood %>%
  filter(restaurant == "Dairy Queen")
```

1. Make a plot (or plots) to visualize the distributions of the amount of calories from fat of the options from these two restaurants. How do their centers, shapes, and spreads compare?

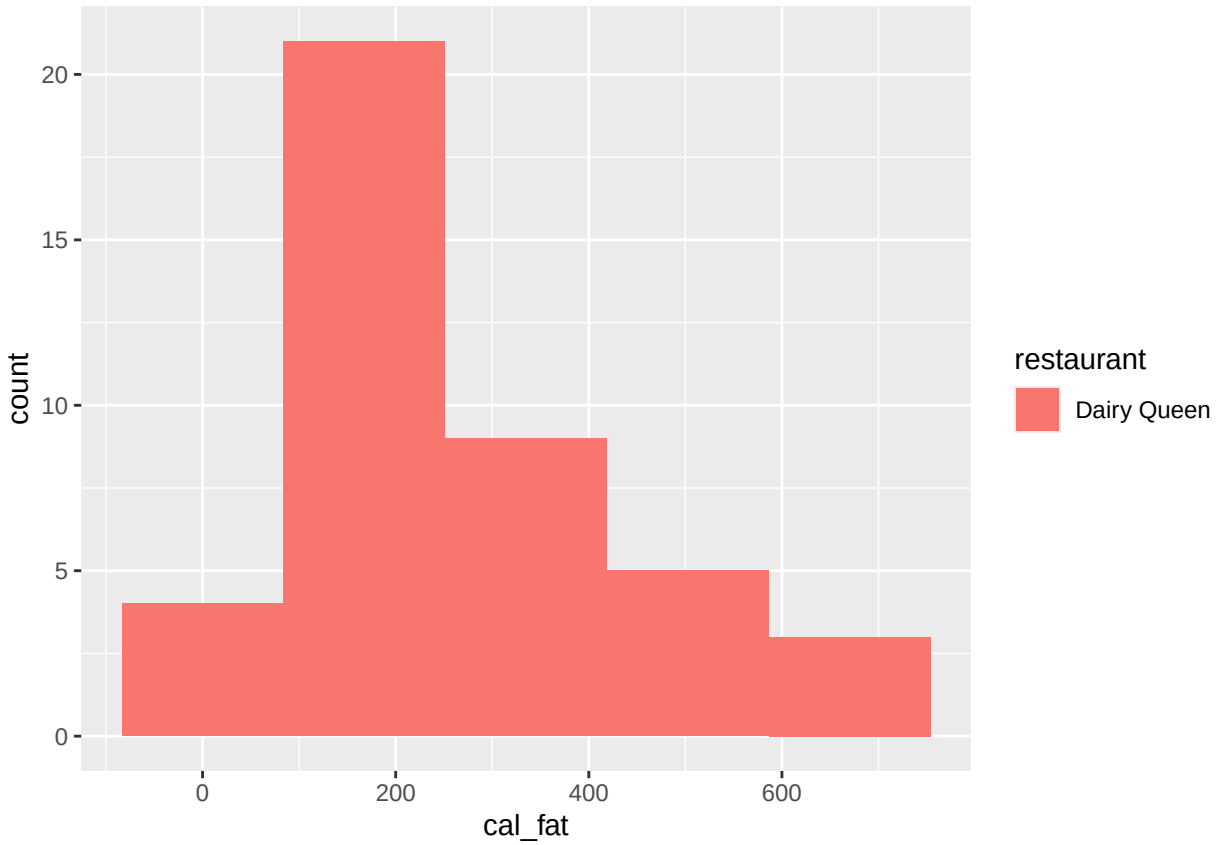
```r
fastfood |>
  filter(restaurant %in% c("Mcdonalds", "Dairy Queen")) |>
  ggplot(aes(x = restaurant, y = cal_fat, fill = restaurant)) +
  geom_point()
```




```r
mcdonalds |>
  ggplot(aes(x = cal_fat, fill = restaurant)) +
  geom_histogram(bins = 5)
```




```
dairy_queen |>
  ggplot(aes(x = cal_fat, fill = restaurant)) +
  geom_histogram(bins = 5)
```



McDonald's has more cal_fat than Dairy Queen, which is evident from the column. From the point we can see that McDonald's is mainly within the 0-500 range with some outliers to 1500 cal_fat.It also looks like McDonald's has more data points than Dairy Queen. If we take a look at the histogram Dairy has a somewhat normal distribution, slightly skewed right, while McDonald's is heavily skewed right.

The normal distribution

In your description of the distributions, did you use words like *bell-shaped*or *normal*? It's tempting to say so when faced with a unimodal symmetric distribution.

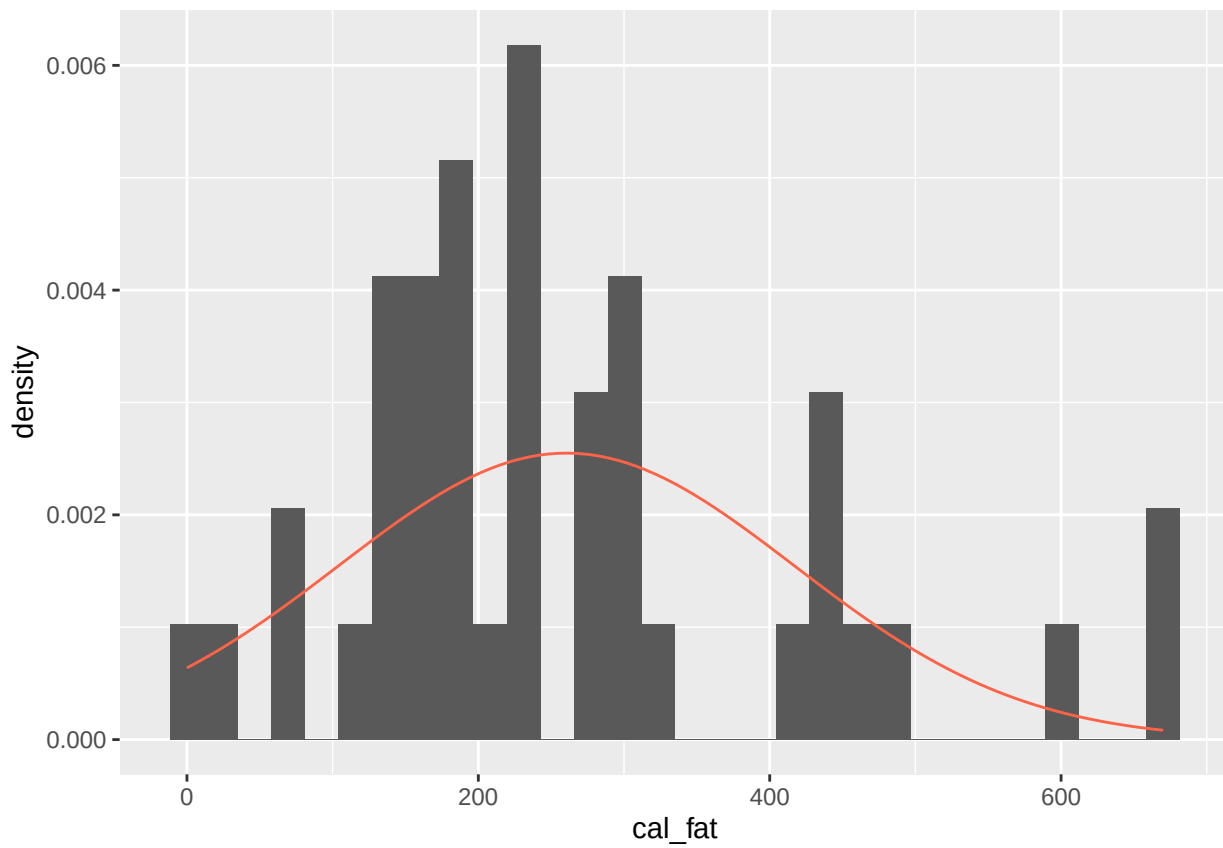
To see how accurate that description is, you can plot a normal distribution curve on top of a histogram to see how closely the data follow a normal distribution. This normal curve should have the same mean and standard deviation as the data. You'll be focusing on calories from fat from Dairy Queen products, so let's store them as a separate object and then calculate some statistics that will be referenced later.

```
dqmean <- mean(dairy_queen$cal_fat)
dqsd   <- sd(dairy_queen$cal_fat)
```

Next, you make a density histogram to use as the backdrop and use the `lines` function to overlay a normal probability curve. The difference between a frequency histogram and a density histogram is that while in a frequency histogram the *heights* of the bars add up to the total number of observations, in a density histogram the *areas* of the bars add up to 1. The area of each bar can be calculated as simply the height *times* the width of the bar. Using a density histogram allows us to properly overlay a normal distribution curve over the histogram since the curve is a normal probability density function that also has area under the curve of 1. Frequency and density histograms both display the same exact shape; they only differ in their

y-axis. You can verify this by comparing the frequency histogram you constructed earlier and the density histogram created by the commands below.

```
ggplot(data = dairy_queen, aes(x = cal_fat)) +
        geom_blank() +
        geom_histogram(aes(y = ..density..)) +
        stat_function(fun = dnorm, args = c(mean = dqmean, sd = dqsd), col = "tomato")
```



After initializing a blank plot with `geom_blank()`, the `ggplot2` package (within the `tidyverse`) allows us to add additional layers. The first layer is a density histogram. The second layer is a statistical function – the density of the normal curve, `dnorm`. We specify that we want the curve to have the same mean and standard deviation as the column of fat calories. The argument `col` simply sets the color for the line to be drawn. If we left it out, the line would be drawn in black.

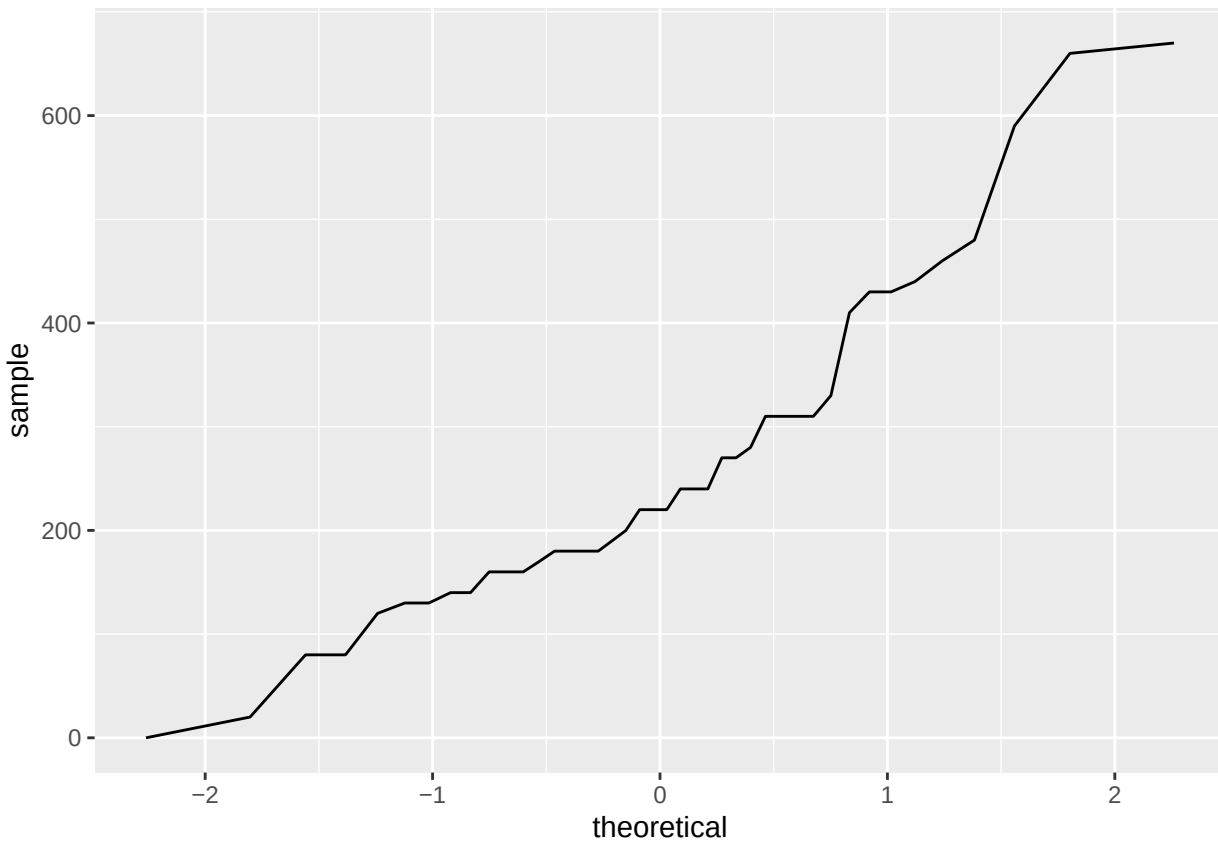
2. Based on the this plot, does it appear that the data follow a nearly normal distribution?

It's slightly skewed right, otherwise it appears it would follow a normal distribution.

Evaluating the normal distribution

Eyeballing the shape of the histogram is one way to determine if the data appear to be nearly normally distributed, but it can be frustrating to decide just how close the histogram is to the curve. An alternative approach involves constructing a normal probability plot, also called a normal Q-Q plot for "quantile-quantile".

```r
ggplot(data = dairy_queen, aes(sample = cal_fat)) +
  geom_line(stat = "qq")
```



This time, you can use the `geom_line()` layer, while specifying that you will be creating a Q-Q plot with the `stat` argument. It's important to note that here, instead of using `x` instead `aes()`, you need to use `sample`.

The x-axis values correspond to the quantiles of a theoretically normal curve with mean 0 and standard deviation 1 (i.e., the standard normal distribution). The y-axis values correspond to the quantiles of the original unstandardized sample data. However, even if we were to standardize the sample data values, the Q-Q plot would look identical. A data set that is nearly normal will result in a probability plot where the points closely follow a diagonal line. Any deviations from normality leads to deviations of these points from that line.

The plot for Dairy Queen's calories from fat shows points that tend to follow the line but with some errant points towards the upper tail. You're left with the same problem that we encountered with the histogram above: how close is close enough?

A useful way to address this question is to rephrase it as: what do probability plots look like for data that I *know* came from a normal distribution? We can answer this by simulating data from a normal distribution using `rnorm`.

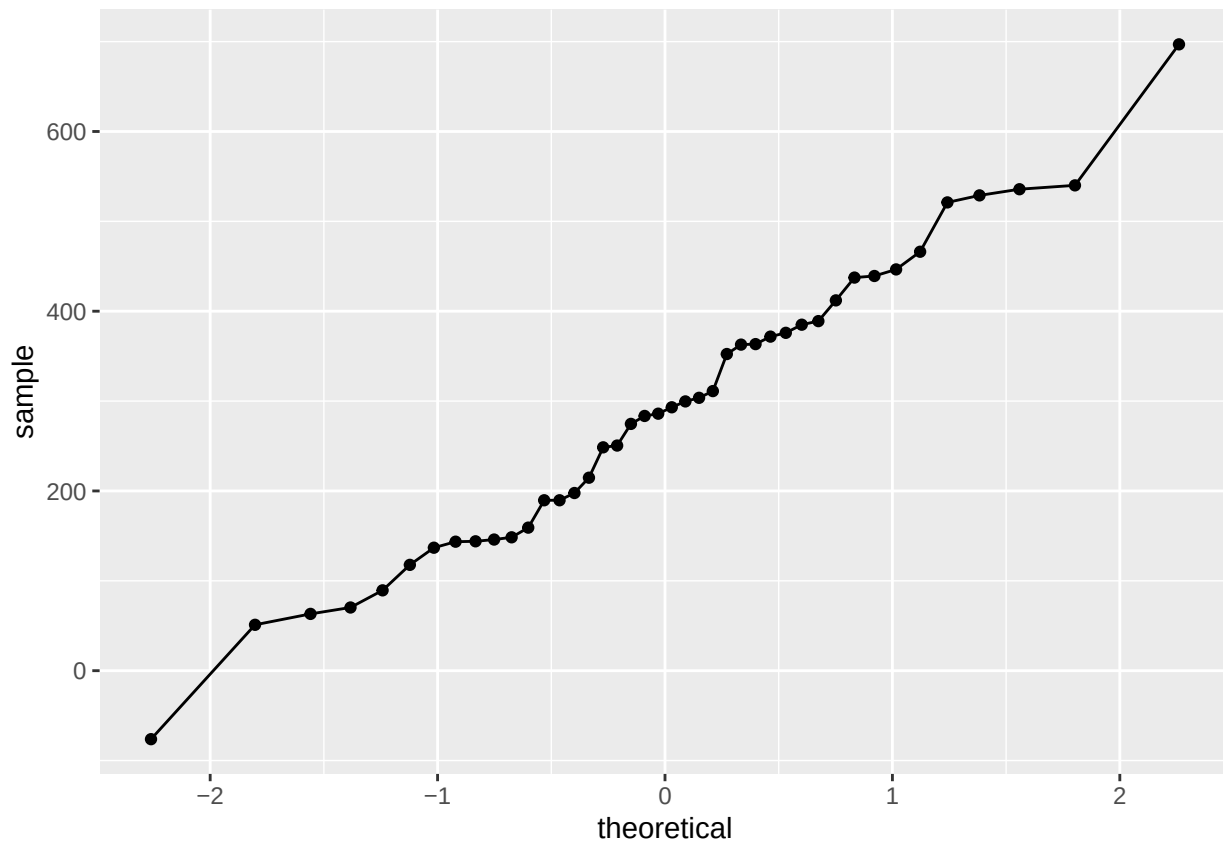
```r
sim_norm <- rnorm(n = nrow(dairy_queen), mean = dqmean, sd = dqsd)
```

The first argument indicates how many numbers you'd like to generate, which we specify to be the same number of menu items in the `dairy_queen` data set using the `nrow()` function. The last two arguments determine the mean and standard deviation of the normal distribution from which the simulated sample will

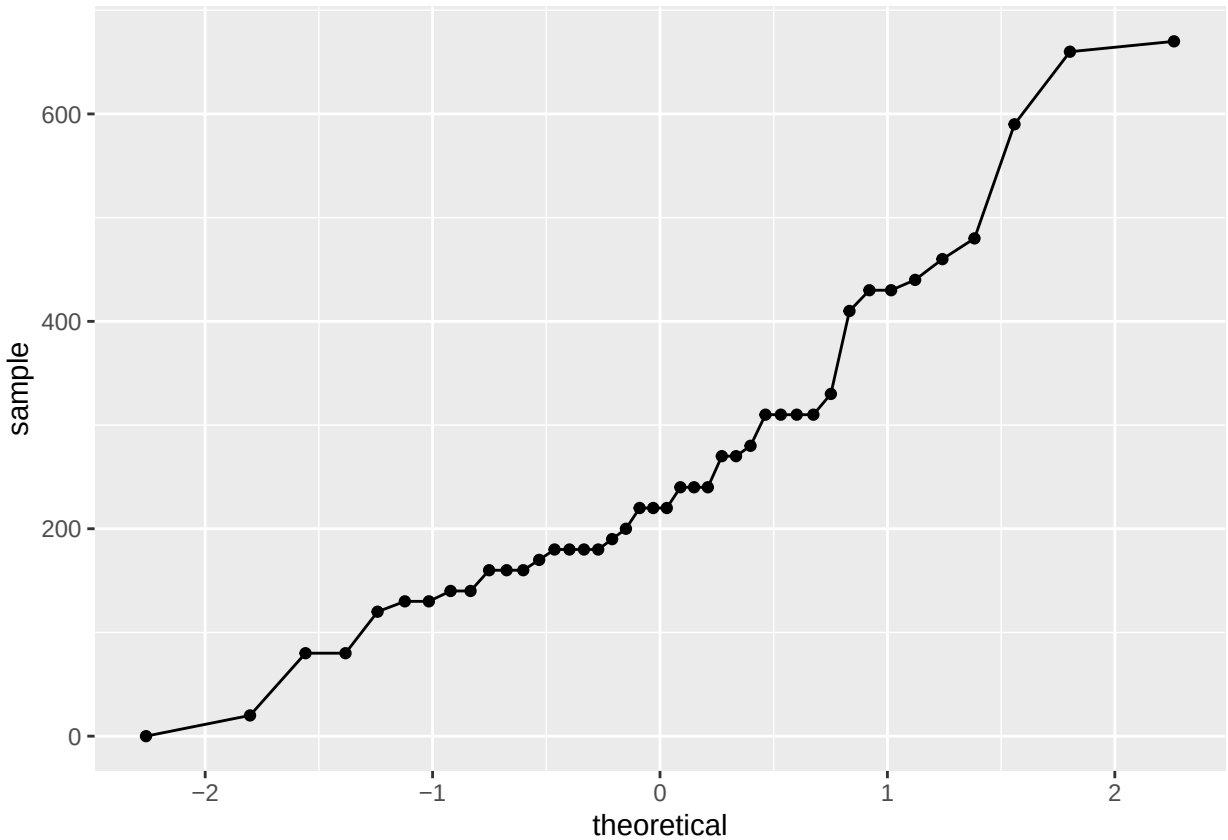
be generated. You can take a look at the shape of our simulated data set, `sim_norm`, as well as its normal probability plot.

3. Make a normal probability plot of `sim_norm`. Do all of the points fall on the line? How does this plot compare to the probability plot for the real data? (Since `sim_norm` is not a data frame, it can be put directly into the `sample` argument and the `data` argument can be dropped.)

```
ggplot(mapping = aes(sample = sim_norm)) +
  geom_line(stat = "qq") +
  geom_point(stat = "qq")
```



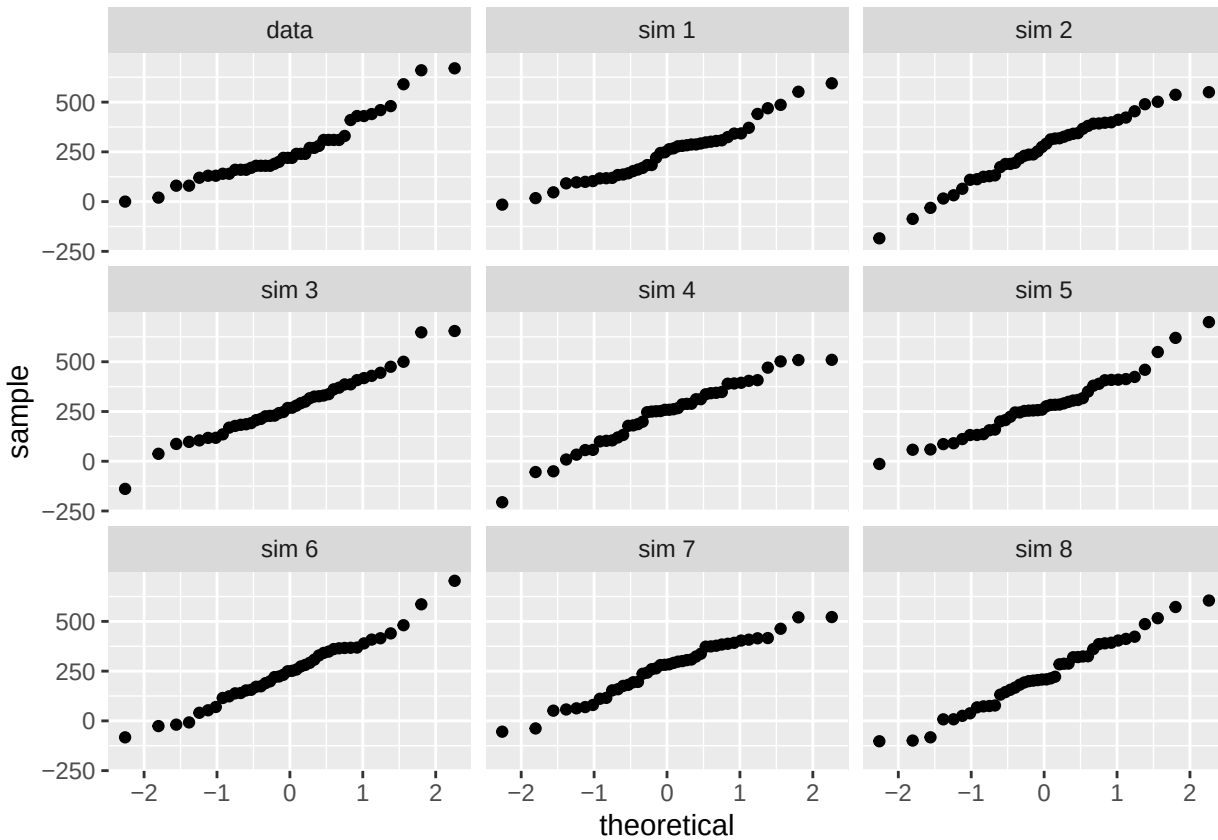

```
ggplot(data = dairy_queen, aes(sample = cal_fat)) +
  geom_line(stat = "qq") +
  geom_point(stat = "qq")
```



Yes, all points fall on the line. I would say that the sim_norm version follows more diagonally than the real data, but they aren't that different.

Even better than comparing the original plot to a single plot generated from a normal distribution is to compare it to many more plots using the following function. It shows the Q-Q plot corresponding to the original data in the top left corner, and the Q-Q plots of 8 different simulated normal data. It may be helpful to click the zoom button in the plot window.

```
qqnormsim(sample = cal_fat, data = dairy_queen)
```



4. Does the normal probability plot for the calories from fat look similar to the plots created for the simulated data? That is, do the plots provide evidence that the calories are nearly normal?

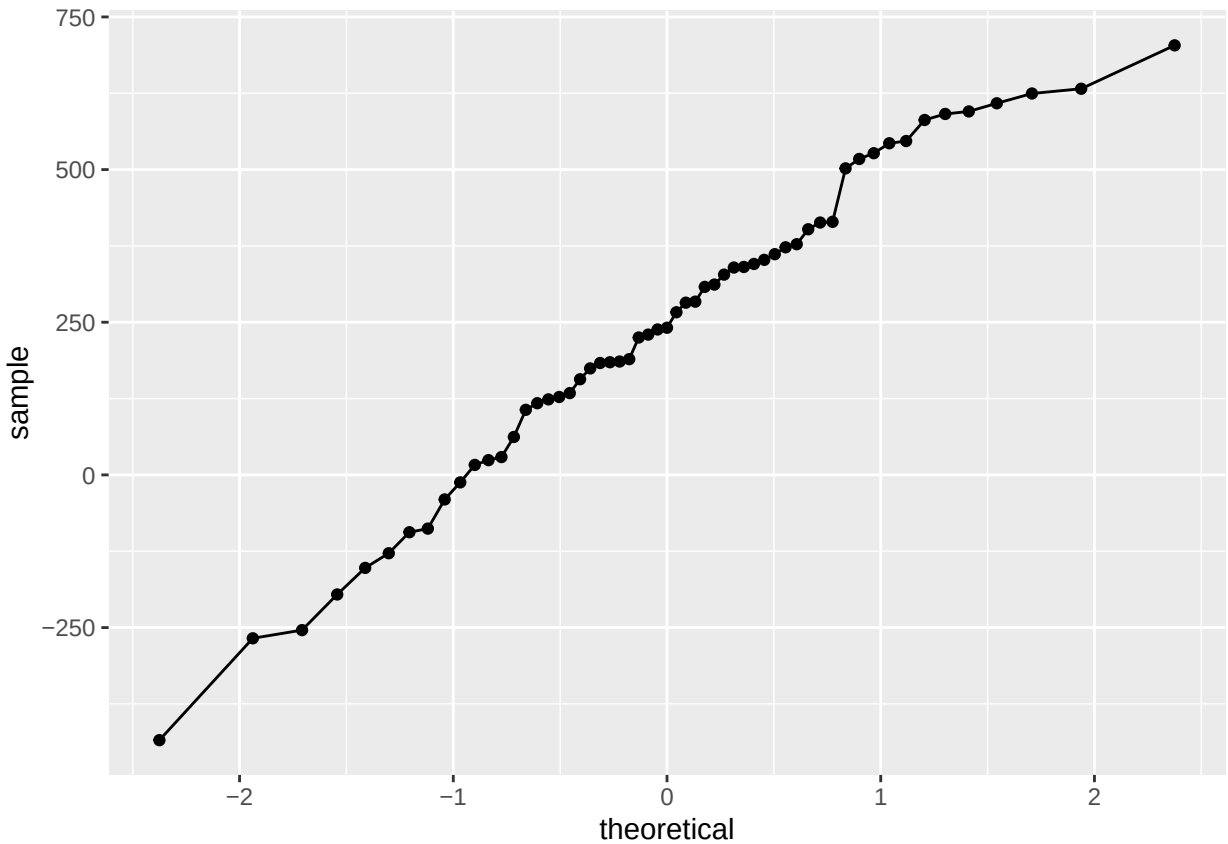
Similar to question 3, I said that they are pretty similar, but the simulated data has more of a normal distribution than the actual data.

5. Using the same technique, determine whether or not the calories from McDonald's menu appear to come from a normal distribution.

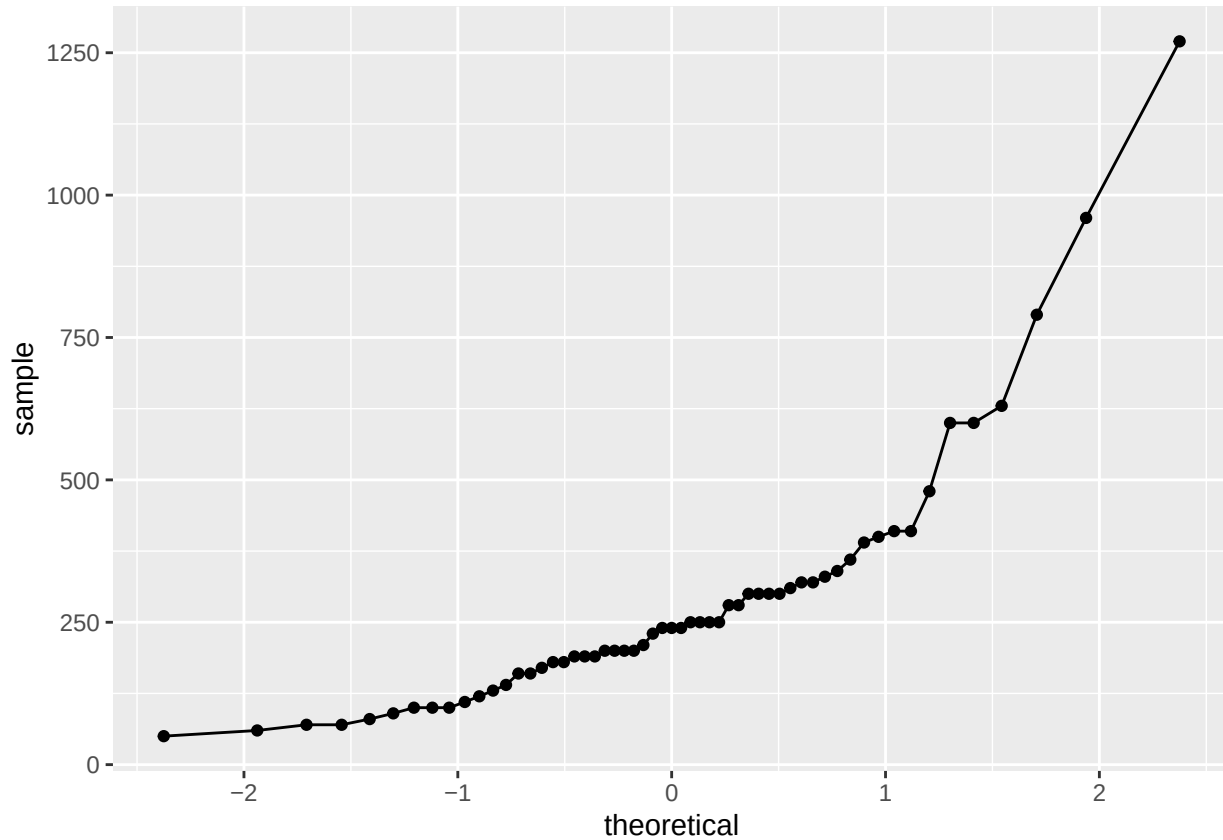
```
mcmean <- mean(mcdonalds$cal_fat)
mcsd   <- sd(mcdonalds$cal_fat)

sim_norm <- rnorm(n = nrow(mcdonalds), mean = mcmean, sd = mcsd)

ggplot(mapping = aes(sample = sim_norm)) +
  geom_line(stat = "qq") +
  geom_point(stat = "qq")
```




```r
ggplot(data = mcdonalds, aes(sample = cal_fat)) +
  geom_line(stat = "qq") +
  geom_point(stat = "qq")
```



Big difference. The simulated data has a nice, nearly, diagonal line, while the actual data is heavily skewed towards 0, almost looks logarithmic.

Normal probabilities

Okay, so now you have a slew of tools to judge whether or not a variable is normally distributed. Why should you care?

It turns out that statisticians know a lot about the normal distribution. Once you decide that a random variable is approximately normal, you can answer all sorts of questions about that variable related to probability. Take, for example, the question of, "What is the probability that a randomly chosen Dairy Queen product has more than 600 calories from fat?"

If we assume that the calories from fat from Dairy Queen's menu are normally distributed (a very close approximation is also okay), we can find this probability by calculating a Z score and consulting a Z table (also called a normal probability table). In R, this is done in one step with the function `pnorm()`.

```
1 - pnorm(q = 600, mean = dqmean, sd = dqsd)
```

```
## [1] 0.01501523
```

Note that the function `pnorm()` gives the area under the normal curve below a given value, `q`, with a given mean and standard deviation. Since we're interested in the probability that a Dairy Queen item has more than 600 calories from fat, we have to take one minus that probability.

Assuming a normal distribution has allowed us to calculate a theoretical probability. If we want to calculate the probability empirically, we simply need to determine how many observations fall above 600 then divide this number by the total sample size.

```
dairy_queen %>%
  filter(cal_fat > 600) %>%
  summarise(percent = n() / nrow(dairy_queen))
```

```
## # A tibble: 1 x 1
##   percent
##     <dbl>
## 1  0.0476
```

Although the probabilities are not exactly the same, they are reasonably close. The closer that your distribution is to being normal, the more accurate the theoretical probabilities will be.

6. Write out two probability questions that you would like to answer about any of the restaurants in this dataset. Calculate those probabilities using both the theoretical normal distribution as well as the empirical distribution (four probabilities in all). Which one had a closer agreement between the two methods?

```
unique(fastfood$restaurant)
```

```
## [1] "Mcdonalds"   "Chick Fil-A" "Sonic"       "Arbys"       "Burger King"
## [6] "Dairy Queen" "Subway"      "Taco Bell"
```

```
chick_fil_a <- fastfood|>
  filter(restaurant == "Chick Fil-A")

cfmean <- mean(chick_fil_a$cal_fat)
cfsd   <- sd(chick_fil_a$cal_fat)

1 - pnorm(q = 600, mean = cfmean, sd = cfsd)
```

```
## [1] 4.46471e-06
```

```
chick_fil_a |>
  filter(cal_fat > 600) |>
  summarize(percent = n() / nrow(chick_fil_a))
```

```
## # A tibble: 1 x 1
##   percent
##     <dbl>
## 1       0
```

```
cfmean <- mean(chick_fil_a$protein)
cfsd   <- sd(chick_fil_a$protein)

1 - pnorm(q = 40, mean = cfmean, sd = cfsd)
```

```
## [1] 0.312024
```

```r
chick_fil_a |>
  filter(protein > 40) |>
  summarize(percent = n() / nrow(chick_fil_a))
```

```
## # A tibble: 1 x 1
##   percent
##     <dbl>
## 1   0.111
```

I viewed the probability of `Chick Fil-A` items having more than 600 cal_fat, also the probability of their items having more than 40g of protein. The cal_fat data ~ theoretical: 4.46471e-06 / empirical: 0 | protein data ~ theoretical: 0.312024 / empirical: 0.111. The cal_fat had a closer agreement because both were essentially 0, way closer than the protein comparison.*

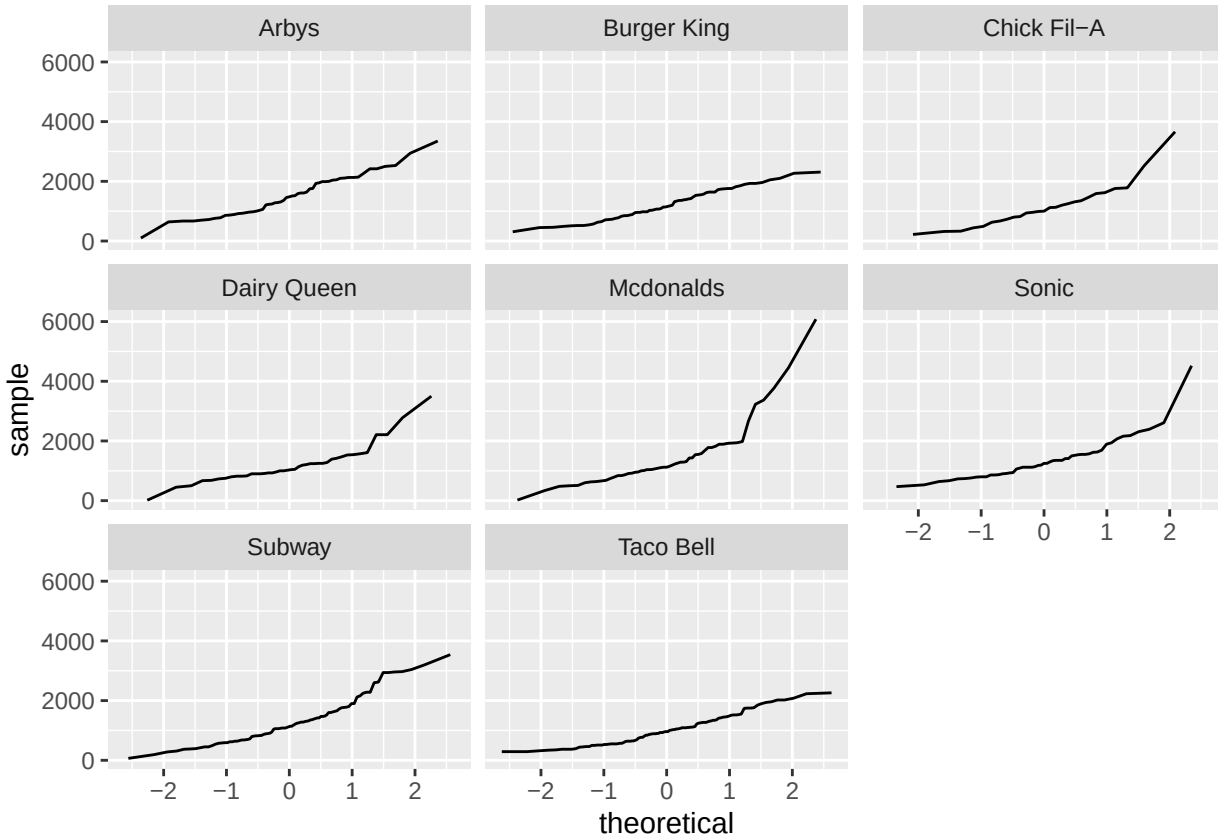
More Practice

7. Now let's consider some of the other variables in the dataset. Out of all the different restaurants, which ones' distribution is the closest to normal for sodium?

```r
glimpse(fastfood)
```

```
## Rows: 515
## Columns: 17
## $ restaurant  <chr> "Mcdonalds", "Mcdonalds", "Mcdonalds", "Mcdonalds", "Mcdon~
## $ item        <chr> "Artisan Grilled Chicken Sandwich", "Single Bacon Smokehou~
## $ calories    <dbl> 380, 840, 1130, 750, 920, 540, 300, 510, 430, 770, 380, 62~
## $ cal_fat     <dbl> 60, 410, 600, 280, 410, 250, 100, 210, 190, 400, 170, 300,~
## $ total_fat   <dbl> 7, 45, 67, 31, 45, 28, 12, 24, 21, 45, 18, 34, 20, 34, 8, ~
## $ sat_fat     <dbl> 2.0, 17.0, 27.0, 10.0, 12.0, 10.0, 5.0, 4.0, 11.0, 21.0, 4~
## $ trans_fat   <dbl> 0.0, 1.5, 3.0, 0.5, 0.5, 1.0, 0.5, 0.0, 1.0, 2.5, 0.0, 1.5~
## $ cholesterol <dbl> 95, 130, 220, 155, 120, 80, 40, 65, 85, 175, 40, 95, 125, ~
## $ sodium      <dbl> 1110, 1580, 1920, 1940, 1980, 950, 680, 1040, 1040, 1290, ~
## $ total_carb  <dbl> 44, 62, 63, 62, 81, 46, 33, 49, 35, 42, 38, 48, 48, 67, 31~
## $ fiber       <dbl> 3, 2, 3, 2, 4, 3, 2, 3, 2, 3, 2, 3, 3, 5, 2, 2, 3, 3, 5, 2~
## $ sugar       <dbl> 11, 18, 18, 18, 18, 9, 7, 6, 7, 10, 5, 11, 11, 11, 6, 3, 1~
## $ protein     <dbl> 37, 46, 70, 55, 46, 25, 15, 25, 25, 51, 15, 32, 42, 33, 13~
## $ vit_a       <dbl> 4, 6, 10, 6, 6, 10, 10, 0, 20, 20, 2, 10, 10, 10, 2, 4, 6,~
## $ vit_c       <dbl> 20, 20, 20, 25, 20, 2, 2, 4, 4, 6, 0, 10, 20, 15, 2, 6, 15~
## $ calcium     <dbl> 20, 20, 50, 20, 20, 15, 10, 2, 15, 20, 15, 35, 35, 35, 4, ~
## $ salad       <chr> "Other", "Other", "Other", "Other", "Other", "Other", "Oth~
```

```r
#thank god facet_wrap
ggplot(data = fastfood, aes(sample = sodium)) +
  geom_line(stat = "qq") +
  facet_wrap(~ restaurant)
```



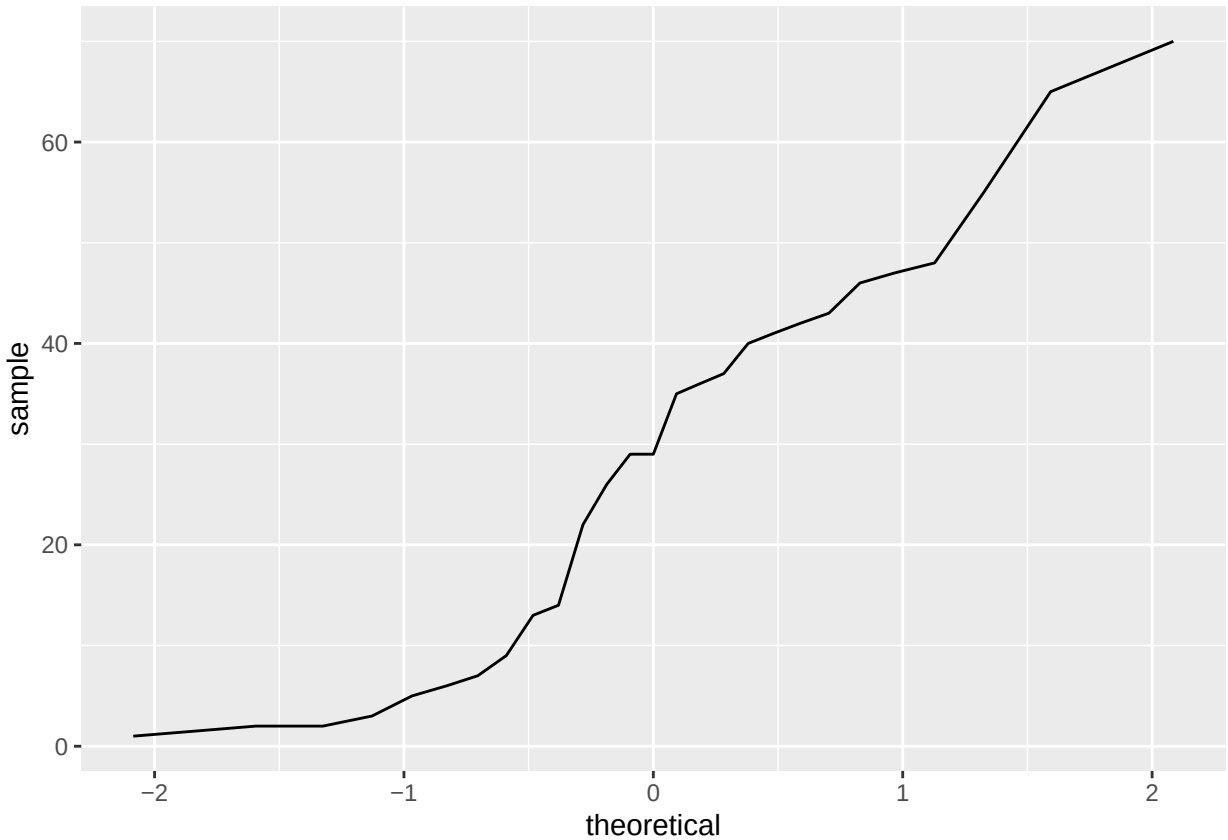
I'd say that Burger King.

8. Note that some of the normal probability plots for sodium distributions seem to have a stepwise pattern. why do you think this might be the case?

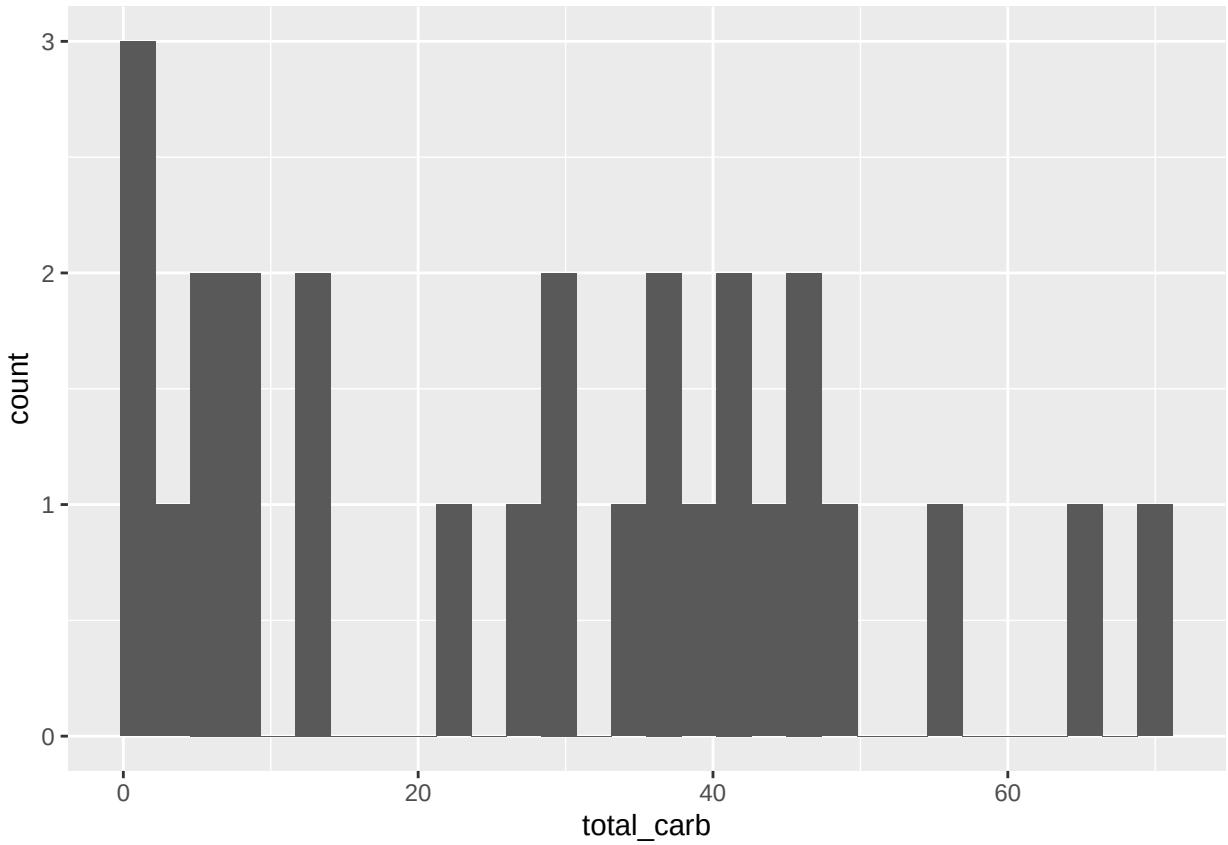
More samples at the same levels of sodium cause the step patterns. The restaurant's case could be be that the fast food restaurant uses items with similar sodium counts or maybe they round their values

9. As you can see, normal probability plots can be used both to assess normality and visualize skewness. Make a normal probability plot for the total carbohydrates from a restaurant of your choice. Based on this normal probability plot, is this variable left skewed, symmetric, or right skewed? Use a histogram to confirm your findings.

```
fastfood |>
  filter(restaurant == "Chick Fil-A") |>
  ggplot(aes(sample = total_carb)) +
  geom_line(stat = "qq")
```




```
fastfood |>
  filter(restaurant == "Chick Fil-A") |>
  ggplot(aes(x = total_carb)) +
  geom_histogram()
```



For Chick Fil-A, total carbs for the probability plot and histogram show a right skew.